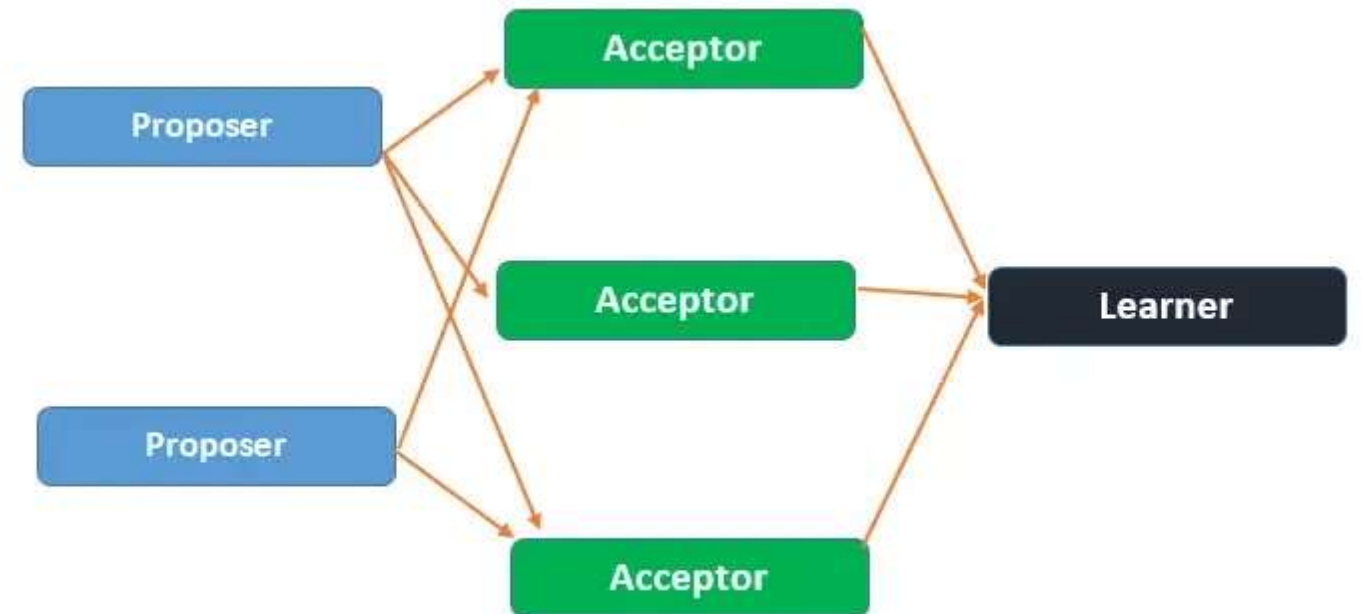


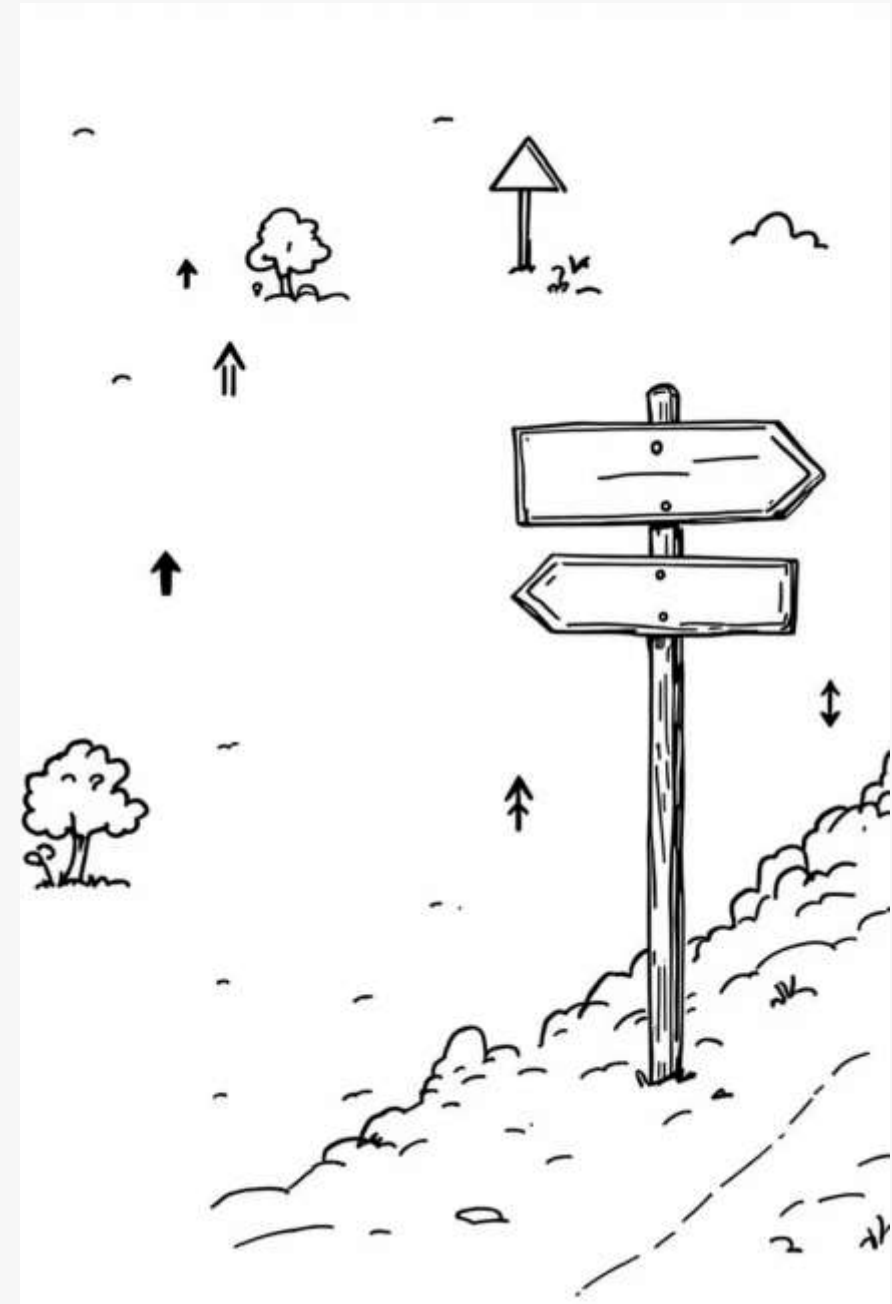
Проектирование Распределенных Систем

Двадцать третий семинар:
Рахос Классический

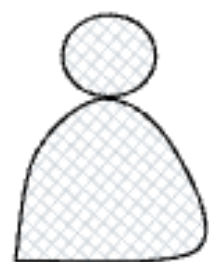


План семинара

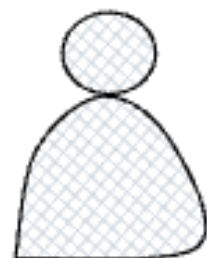
- Проблематика достижения консенсуса в РС
- Свойства алгоритмов достижения консенсуса
- Историческая вводная в Paxos
- Роли в алгоритме Paxos
- Действия каждой роли в Paxos
- Этапы в алгоритме Paxos
- Кворумы в алгоритме Paxos
- Сценарии отказов нод на разных этапах
- Случай с несколькими proposer-ами и борьбой за первенство



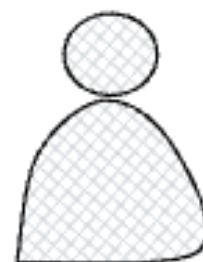
Ситуация



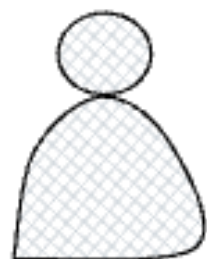
You



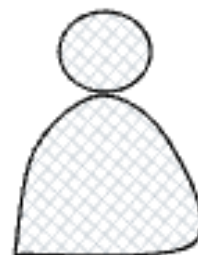
John



Max

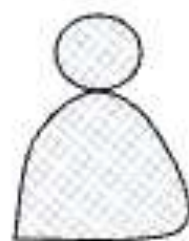


Jane

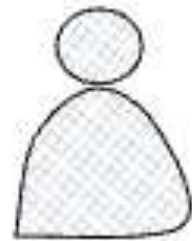


Victoria

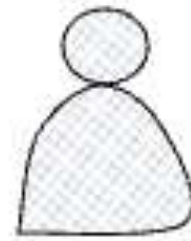
Ситуация



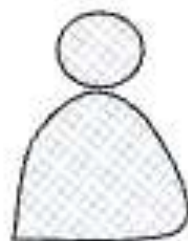
You



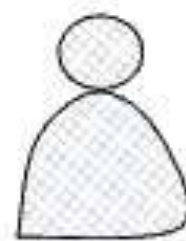
John



Max



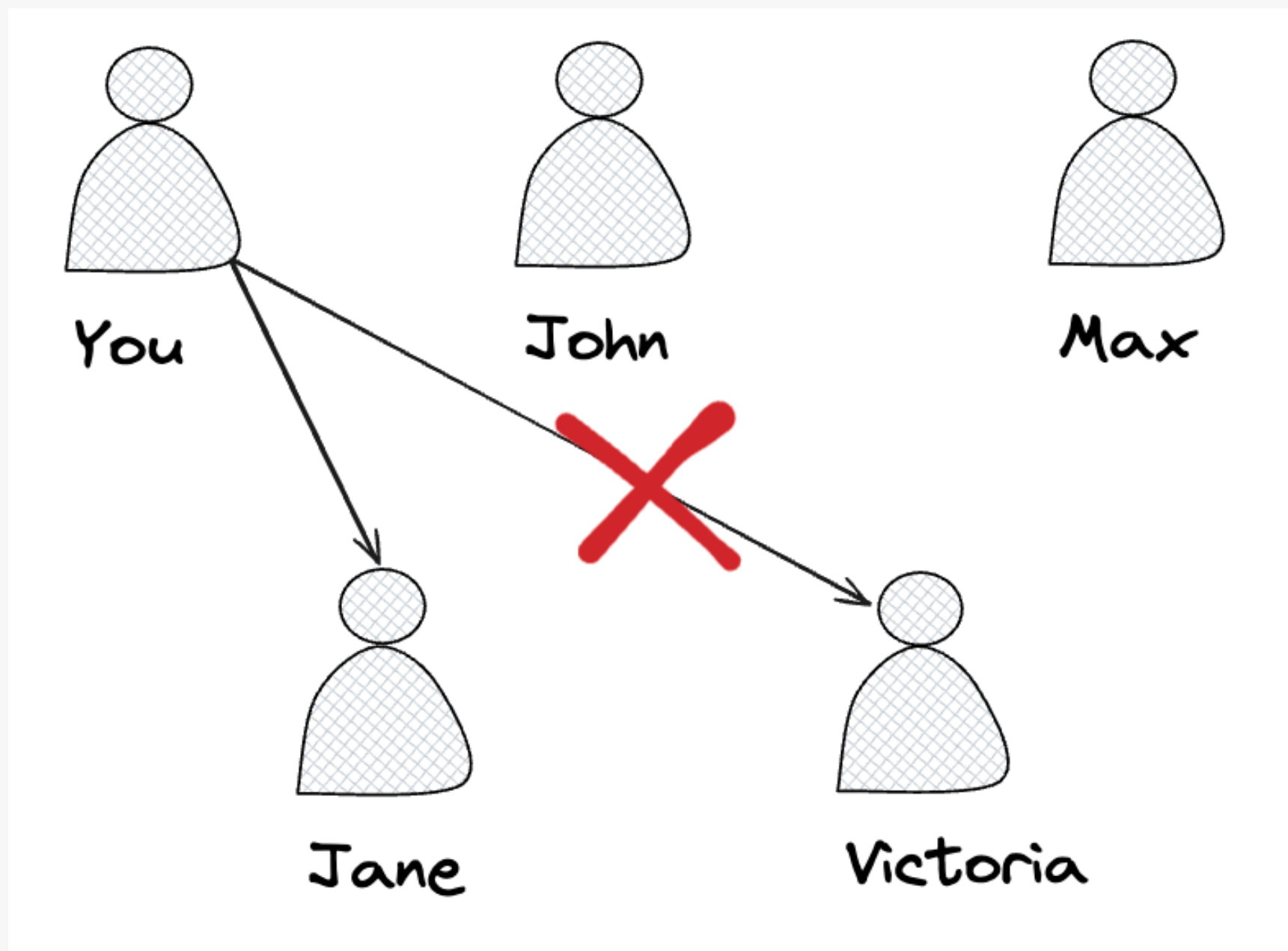
Jane



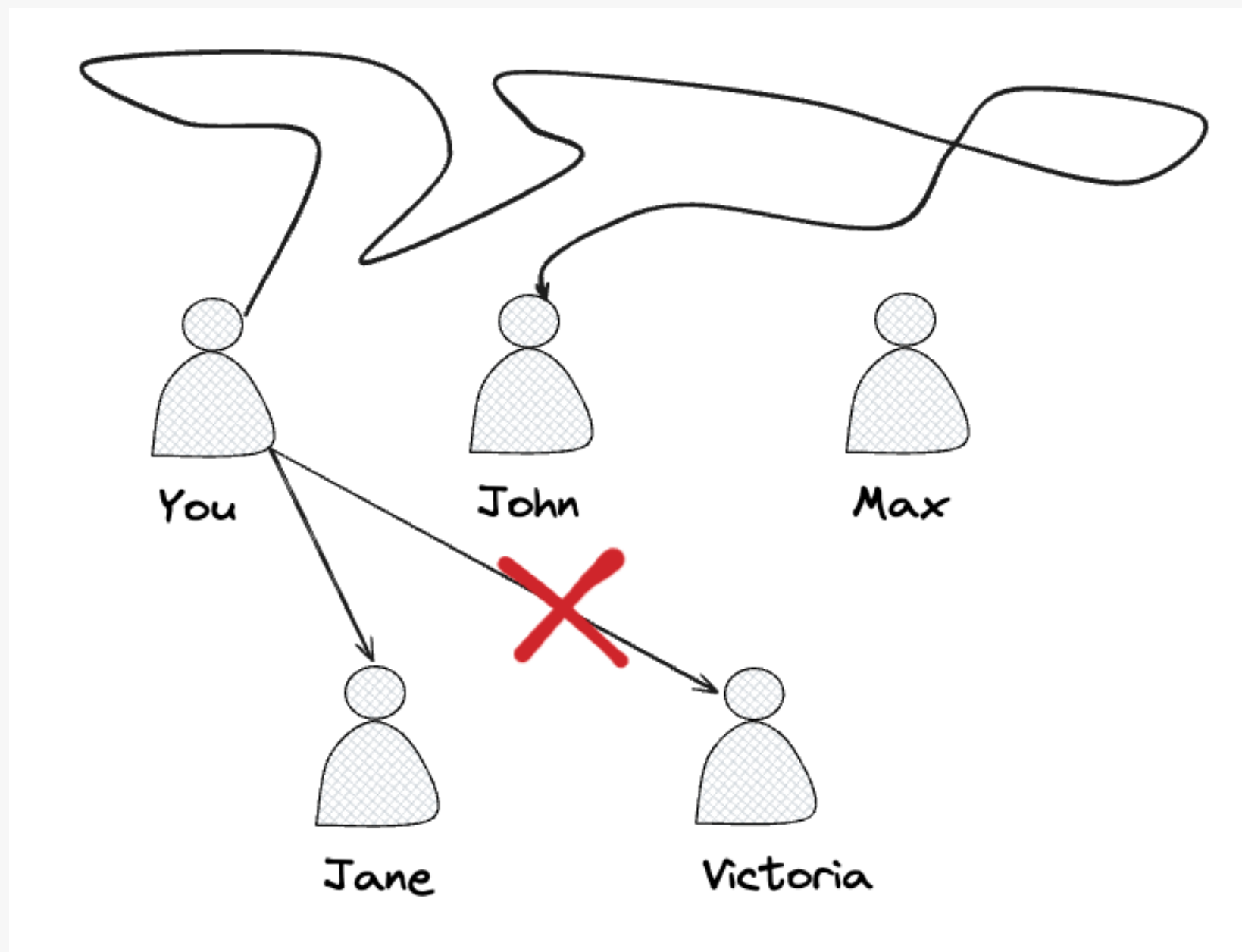
Victoria



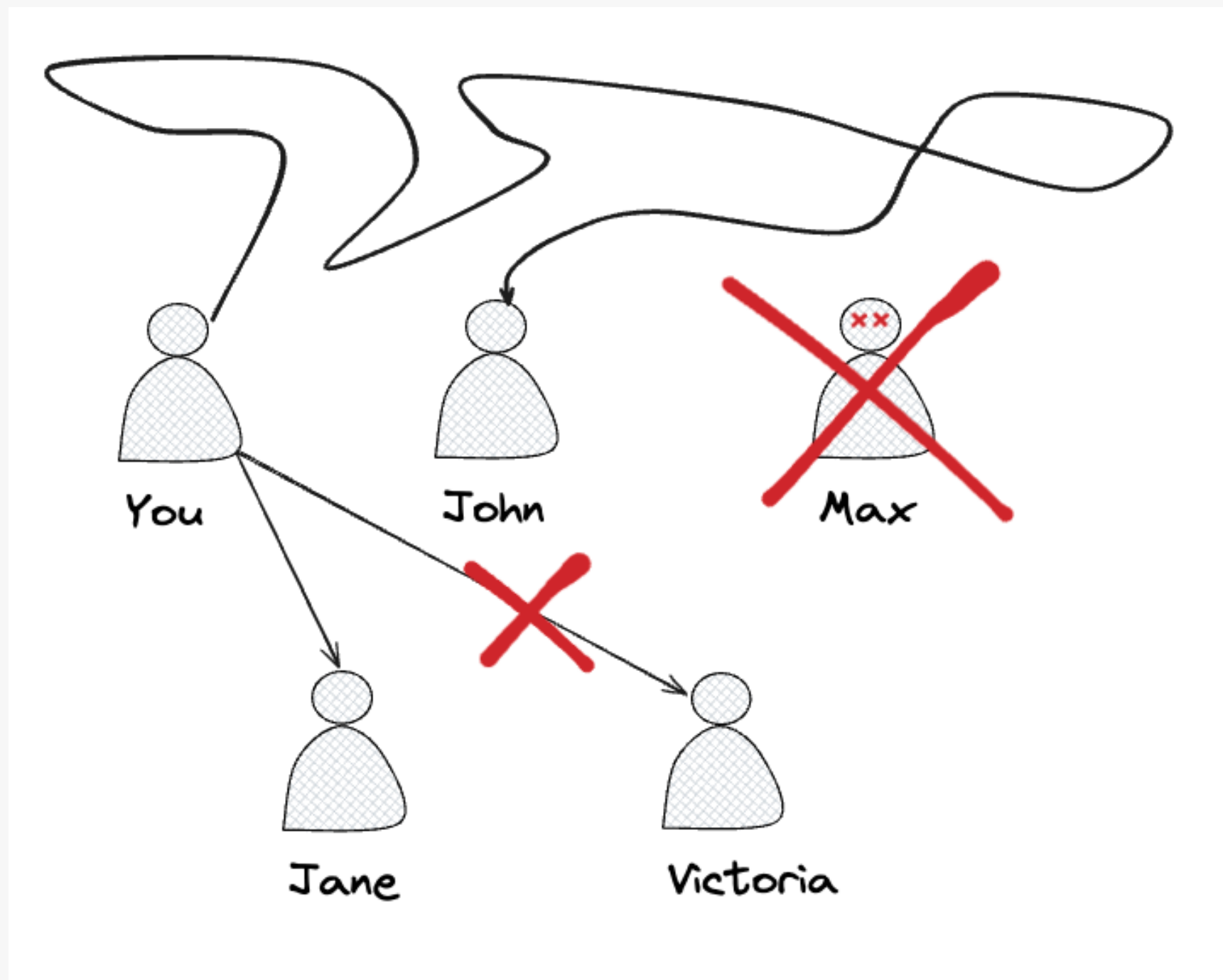
Ситуация



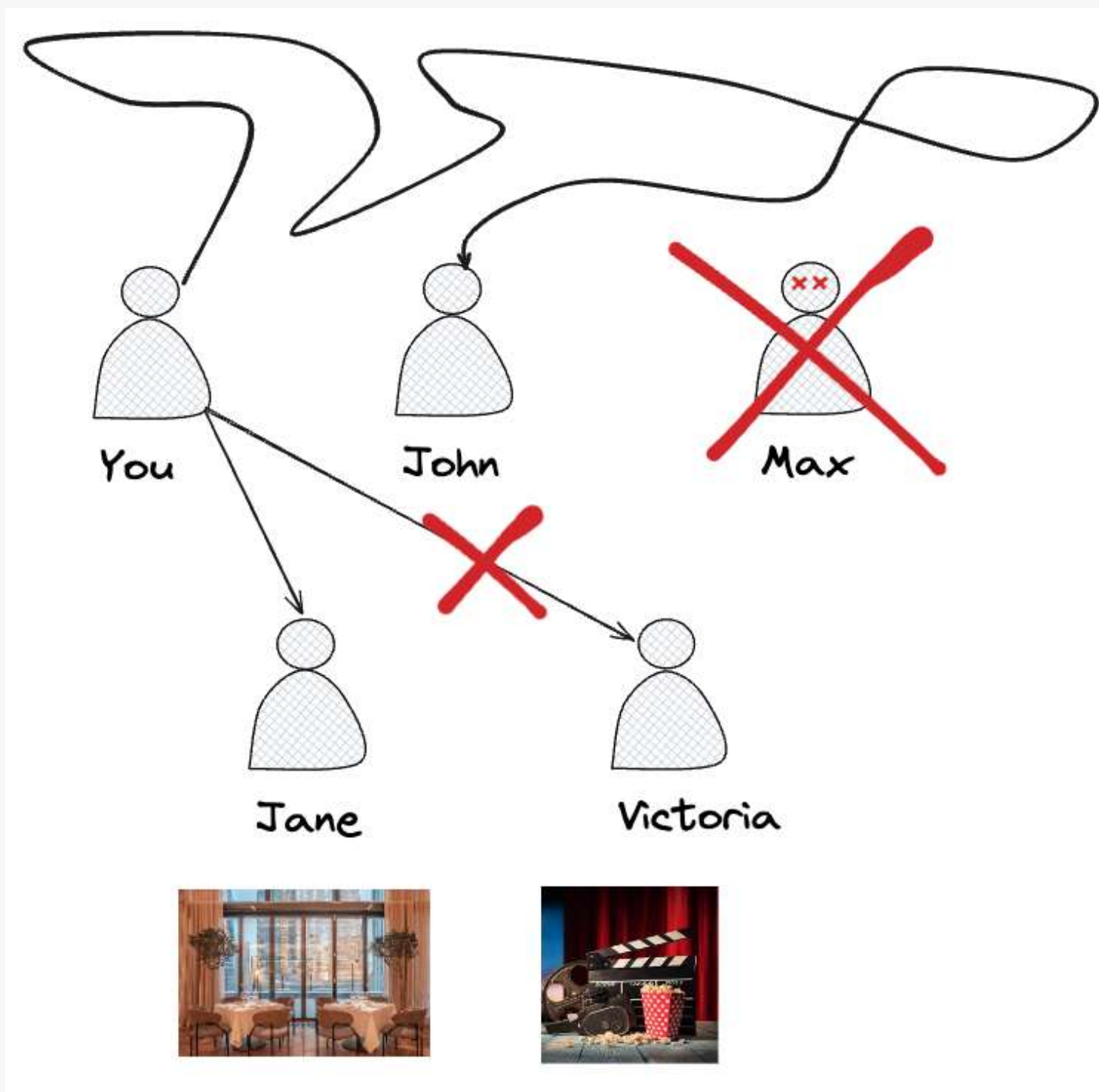
Ситуация



Ситуация



Ситуация

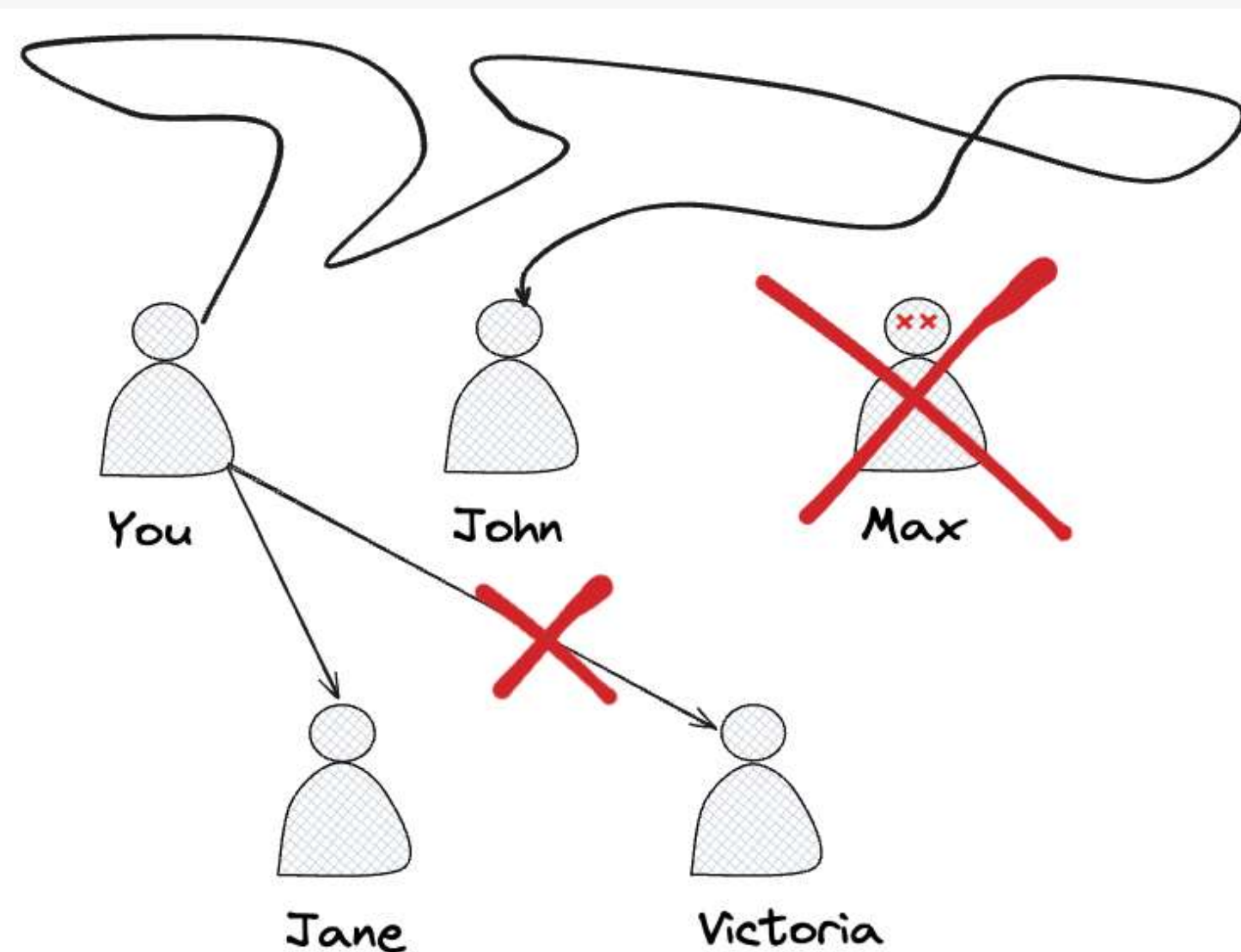


Ситуация

Необходимо придумать способ такой, чтобы:

- все, кто дожил до вечера, узнали ОДНО и то же место
- это место было реально кем-то предложено (а не "встретимся на луне")
- рано или поздно решение все-таки было принято

Это и будет означать достижение консенсуса



Свойства алгоритмов достижения консенсуса

Свойства алгоритмов
достижения консенсуса

Agreement

Все корректные процессы
решают одно и то же
значение

Validity

Выбранное значение
должно быть кем-то
реально предложено.

Termination

Все корректные процессы
рано или поздно
принимают решение.

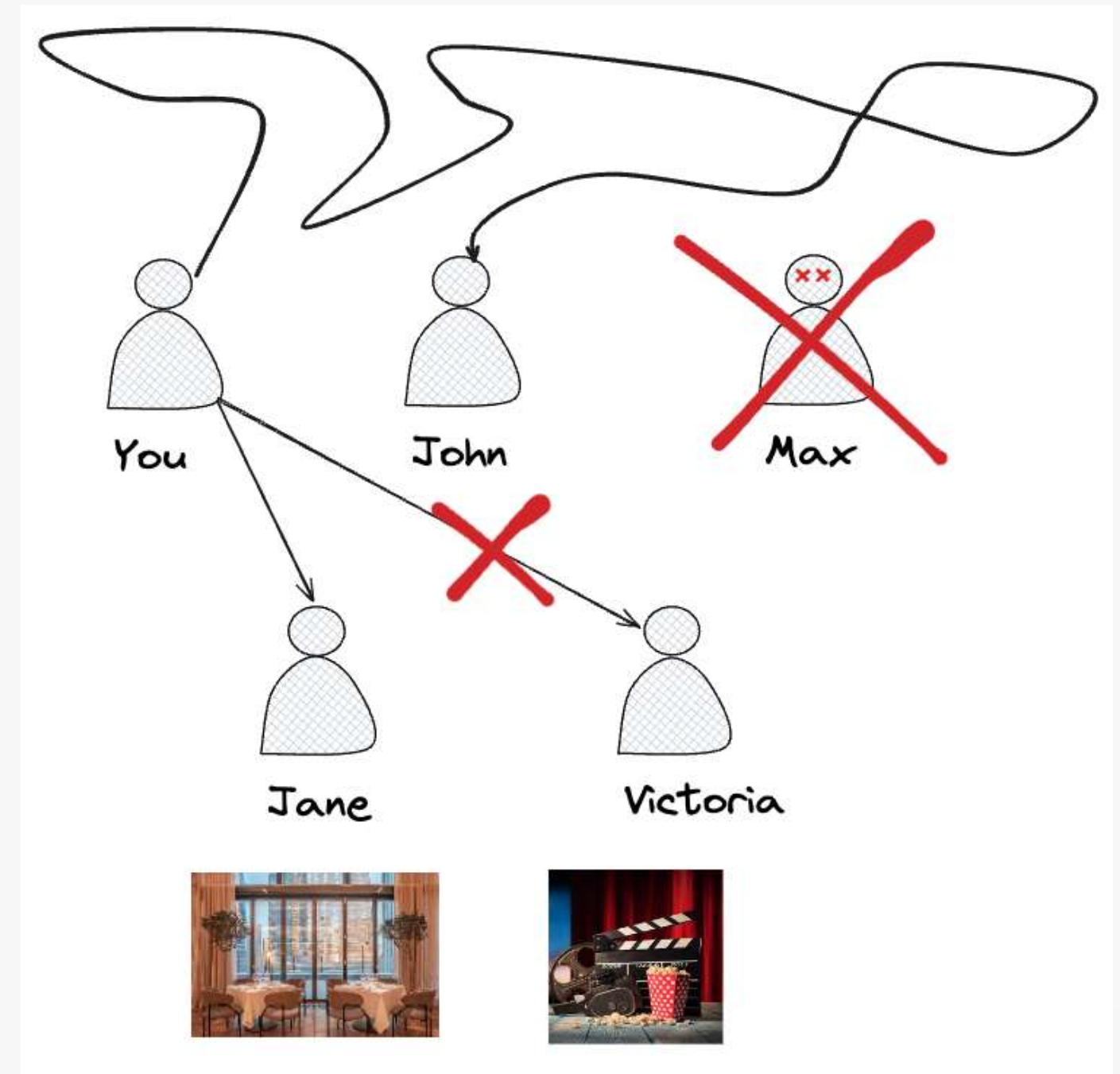
Свойства алгоритмов достижения консенсуса

Свойства:

1. Согласованность (Agreement) — все корректные процессы решают одно и то же значение.

2. Обоснованность (Validity) — выбранное значение должно быть кем-то реально предложено.

3. Завершаемость (Termination) — все корректные процессы рано или поздно принимают решение.

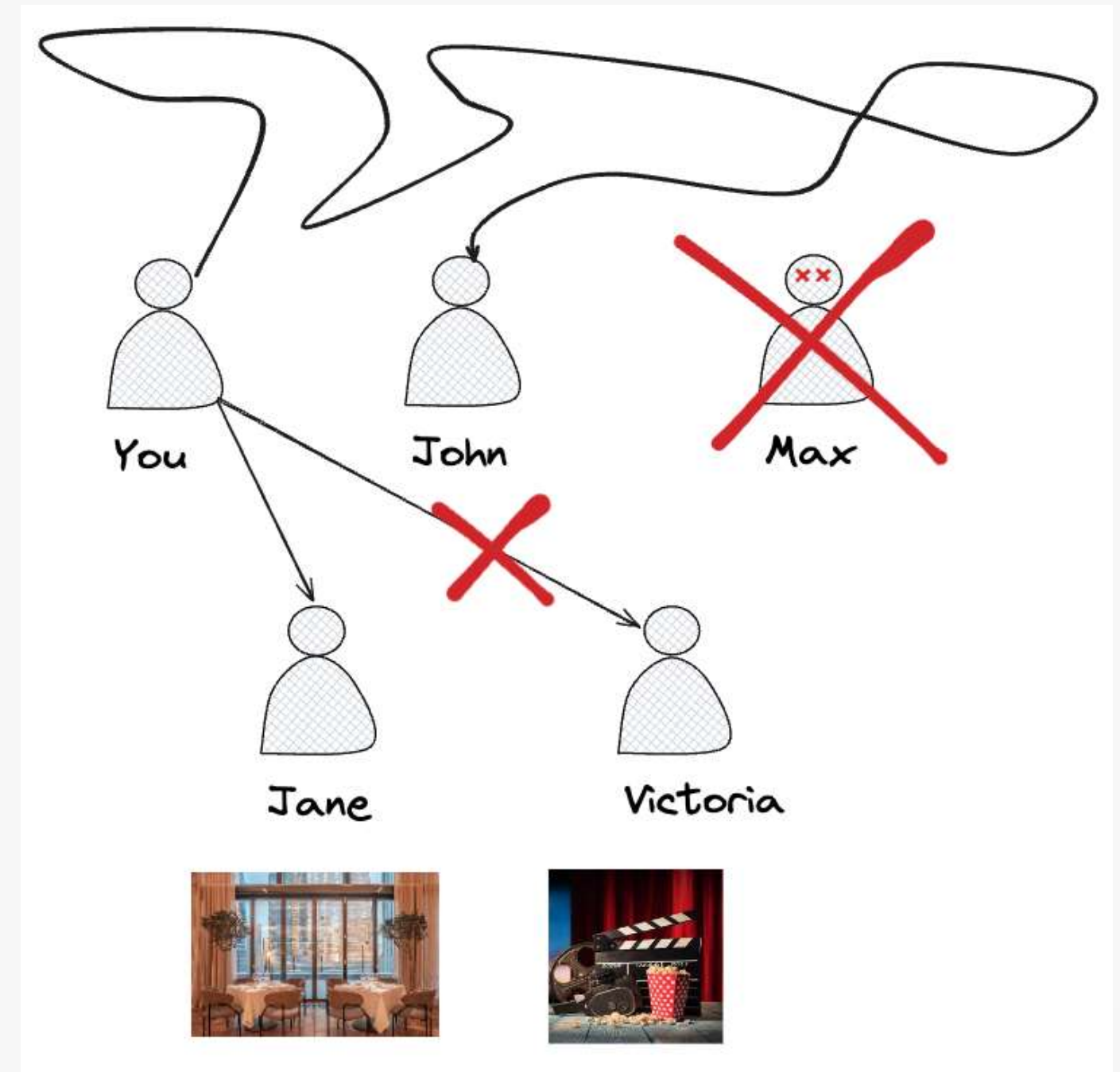


Свойства алгоритмов достижения консенсуса

Для решения как раз такой проблемы был придуман алгоритм (семейство алгоритмов) **Paxos**

Свойства:

1. Согласованность (Agreement) — все корректные процессы решают одно и то же значение.
2. Обоснованность (Validity) — выбранное значение должно быть кем-то реально предложено.
3. Завершаемость (Termination) — все корректные процессы рано или поздно принимают решение.



Историческая вводная про Paxos

История публикации алгоритма Paxos:

- Придумал Leslie Lamport
(на основе работ Lynch-а и Liskov) в 1989 году
- Опубликовал лишь в 1998 году под видом сказки про короткоправящих парламентариев на группе островов Paxos (“The Part Time Parliament”)
- Лишь в 2001-ом году опубликовал упрощенную версию в более научном стиле “Paxos Made Simple”



Рaхoс и его свойства

Рaхoс и его свойства

Agreement

Все корректные процессы
решают одно и то же
значение

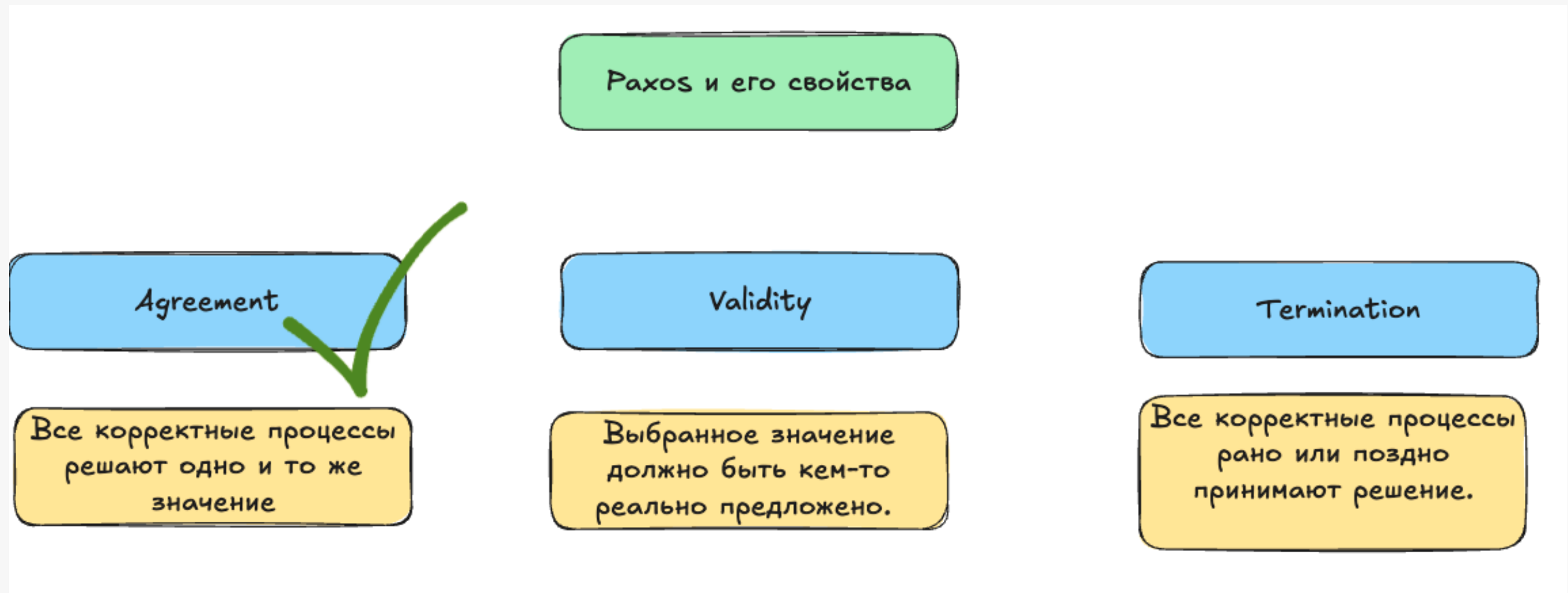
Validity

Выбранное значение
должно быть кем-то
реально предложено.

Termination

Все корректные процессы
рано или поздно
принимают решение.

Рaхoс и его свойства



Ракос и его свойства

Ракос и его свойства

Agreement

Все корректные процессы
решают одно и то же
значение

Validity

Выбранное значение
должно быть кем-то
реально предложено.

Termination

Все корректные процессы
рано или поздно
принимают решение.

Ракос и его свойства

Ракос и его свойства

Agreement

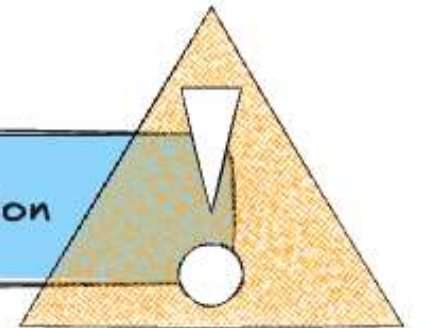
Все корректные процессы
решают одно и то же
значение

Validity

Выбранное значение
должно быть кем-то
реально предложено.

Termination

Все корректные процессы
рано или поздно
принимают решение.



Роли в Paxos

Роли в алгоритме Paxos

Роли в Paxos

Роли в алгоритме Paxos

Proposer

Получает значение,
предлагает его остальным,
борется за то, чтобы его
предложение приняли

Роли в Paxos

Роли в алгоритме Paxos

Proposer

Получает значение,
предлагает его остальным,
борется за то, чтобы его
предложение приняли

Acceptor

Голосует «за/против»
предложений.
Это "хранитель голосов";
от его памяти зависит
safety

Роли в Paxos

Роли в алгоритме Paxos

Proposer

Получает значение,
предлагает его остальным,
борется за то, чтобы его
предложение приняли

Acceptor

Голосует «за/против»
предложений.
Это "хранитель голосов";
от его памяти зависит
safety

Learner

Узнаёт результат и
применяет его к своей
реплике

Роли в Paxos

ВАЖНО:

- Каждая нода может принимать одни ИЛИ БОЛЕЕ ролей одновременно

Роли в алгоритме Paxos

Proposer

Получает значение,
предлагает его остальным,
борется за то, чтобы его
предложение приняли

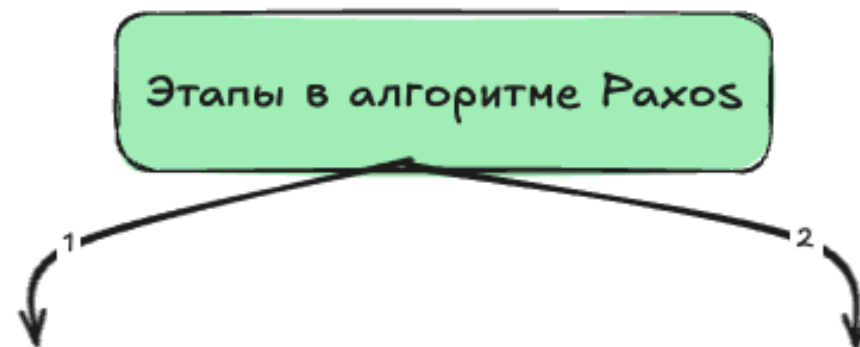
Acceptor

Голосует «за/против»
предложений.
Это "хранитель голосов";
от его памяти зависит
safety

Learner

Узнаёт результат и
применяет его к своей
реплике

Этапы в Paxos



Этапы в Paxos

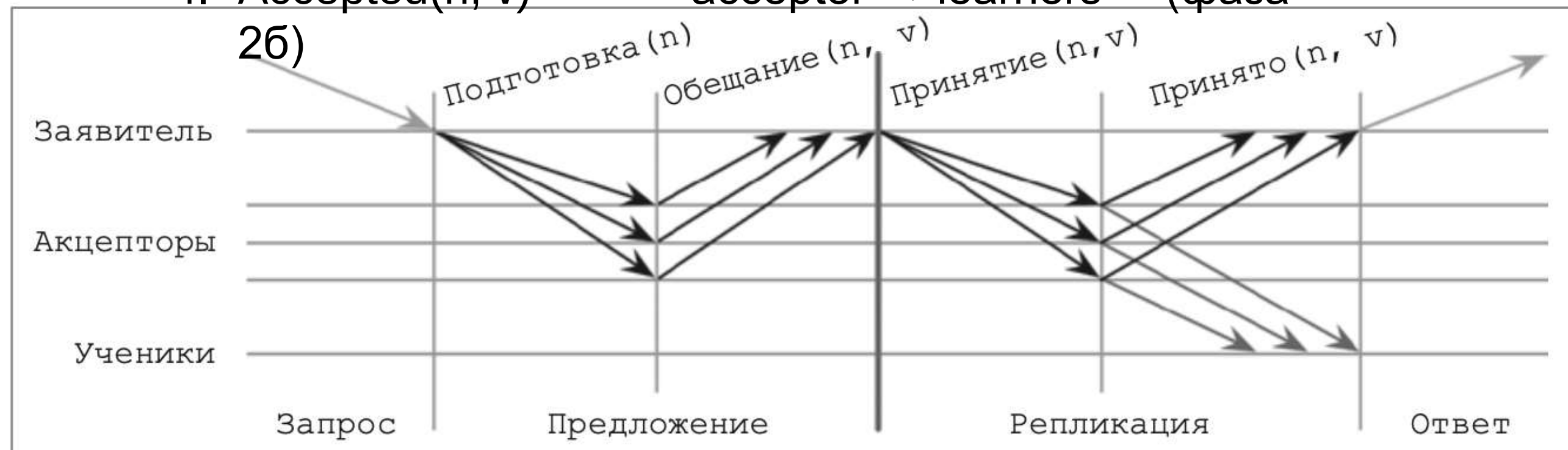


Этапы в Paxos



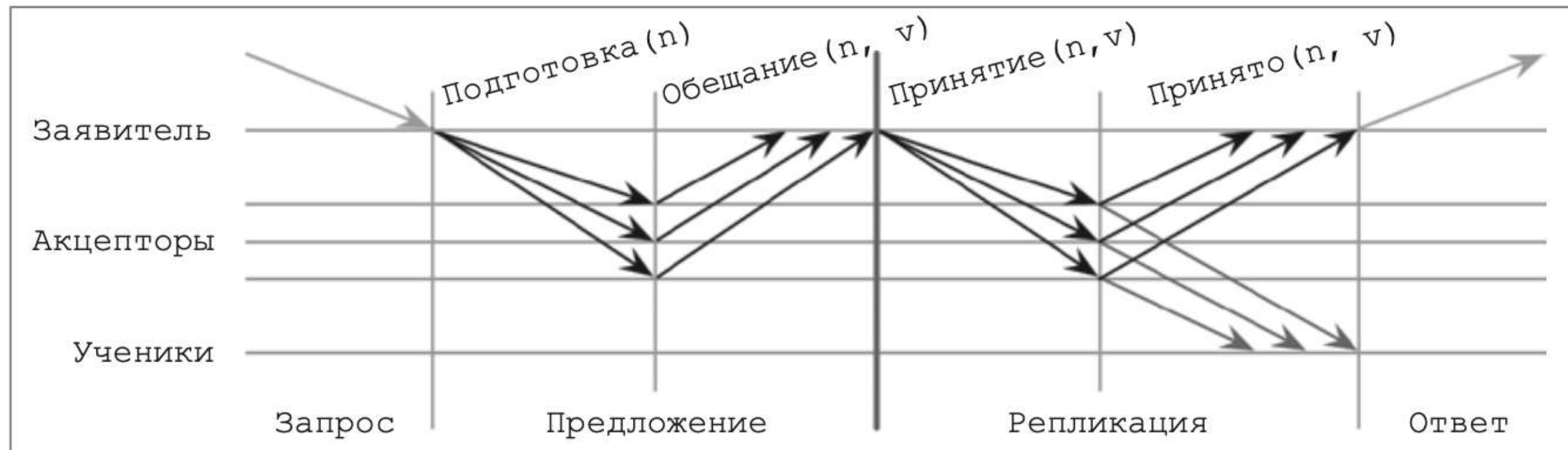
Разбор этапов алгоритма

1. Prepare(n) — proposer → acceptors (фаза 1a)
2. Promise(n, v_a) — acceptor → proposer (фаза 1б)
3. Accept!(n, v) — proposer → acceptors (фаза 2a)
4. Accepted(n, v) — acceptor → learners (фаза 2б)



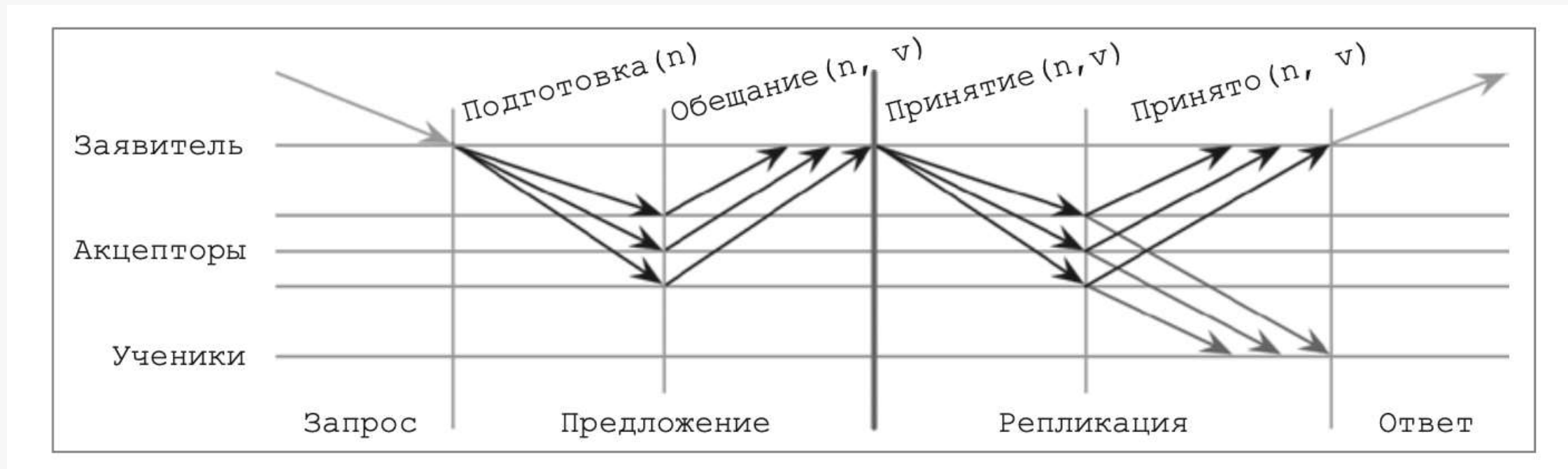
Разбор этапов алгоритма. Proposer stage 1

- Что делает proposer?
- Выбирает свежий номер n больше всего, что он раньше отправлял, и шлёт `Prepare(n)` большинству акцепторов.



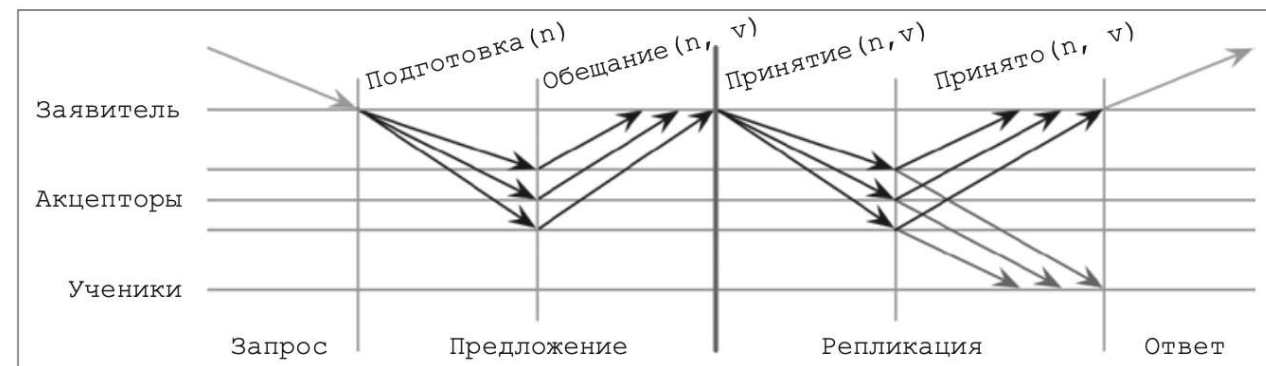
Разбор этапов алгоритма. Acceptor stage 1

- Что делает акцептор, получив `Prepare(n)`?
- Смотрит на свою память — а в памяти у него две вещи:
 1. максимальный номер, на который он уже обещал (`n_promised`)
 2. последнее принятое предложение (`n_accepted, v_accepted`).



Разбор этапов алгоритма. Proposer stage 1 ending

1. Proposer ждёт `Promise` от **большинства**.
Как только большинство собрано —
он переходит к фазе 2.

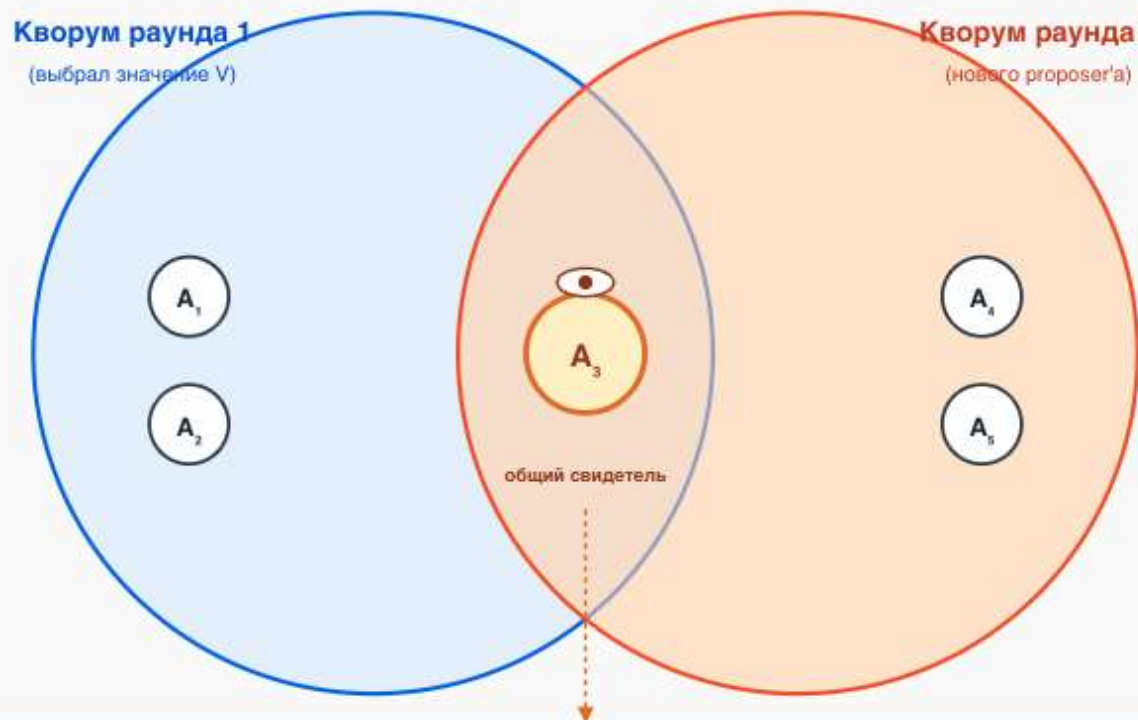


Сердце safety: пересечение большинства

Любые два кворума из $N=5$ имеют хотя бы один общий элемент

Кворум раунда 1
(выбрал значение V)

Кворум раунда 2
(нового proposer'a)



A_3 в Promise сообщает: «у меня уже принято V с номером n_a »

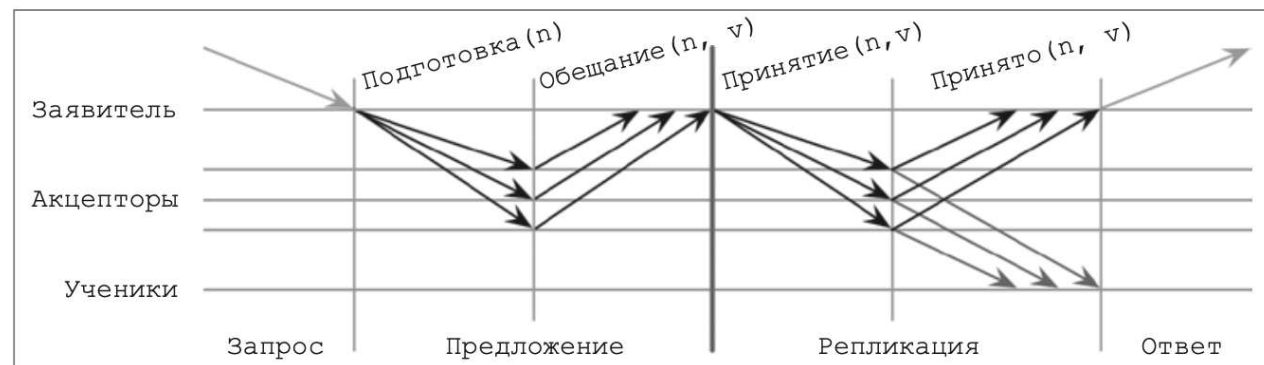
⇒ новый proposer ОБЯЗАН предложить V (правило $\max-n_a$)

⇒ Agreement сохранён ✓

Safety = пересечение большинства + правило $\max-n_a$

Разбор этапов алгоритма. Proposer stage 2

1. Proposer смотрит значения в ответах у большинства.
2. Выбирает из всех значений v_a CAM
3. И далее предлагает уже это v_a_m на следующий этап
4. И только если все promise-ы пустые может предложить свое значение

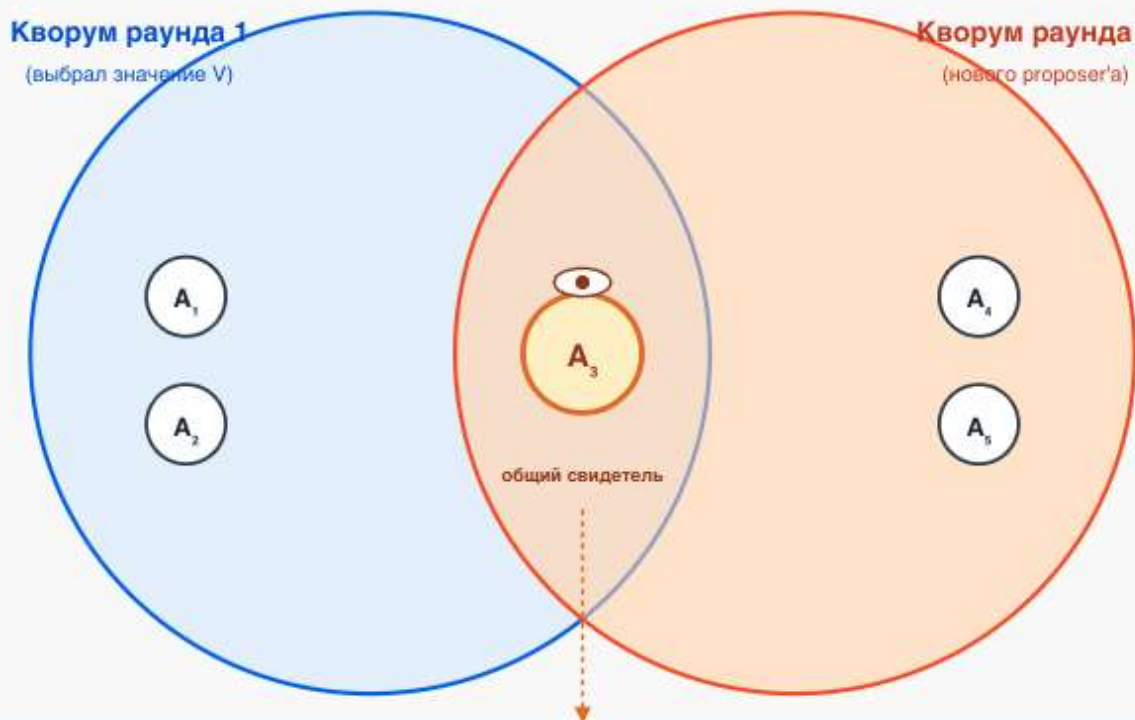


Сердце safety: пересечение большинств

Любые два кворума из $N=5$ имеют хотя бы один общий элемент

Кворум раунда 1
(выбрал значение V)

Кворум раунда 2
(нового proposer'a)



A_3 в Promise сообщает: «у меня уже принято V с номером n_a »

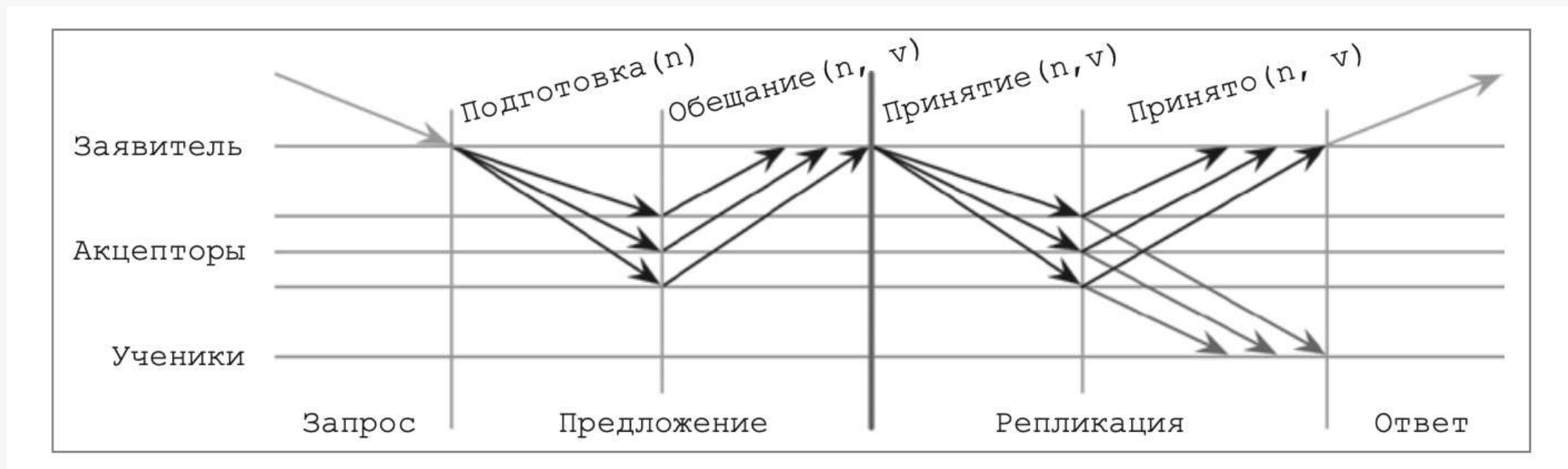
⇒ новый proposer ОБЯЗАН предложить V (правило $\max-n_a$)

⇒ Agreement сохранён ✓

Safety = пересечение большинств + правило $\max-n_a$

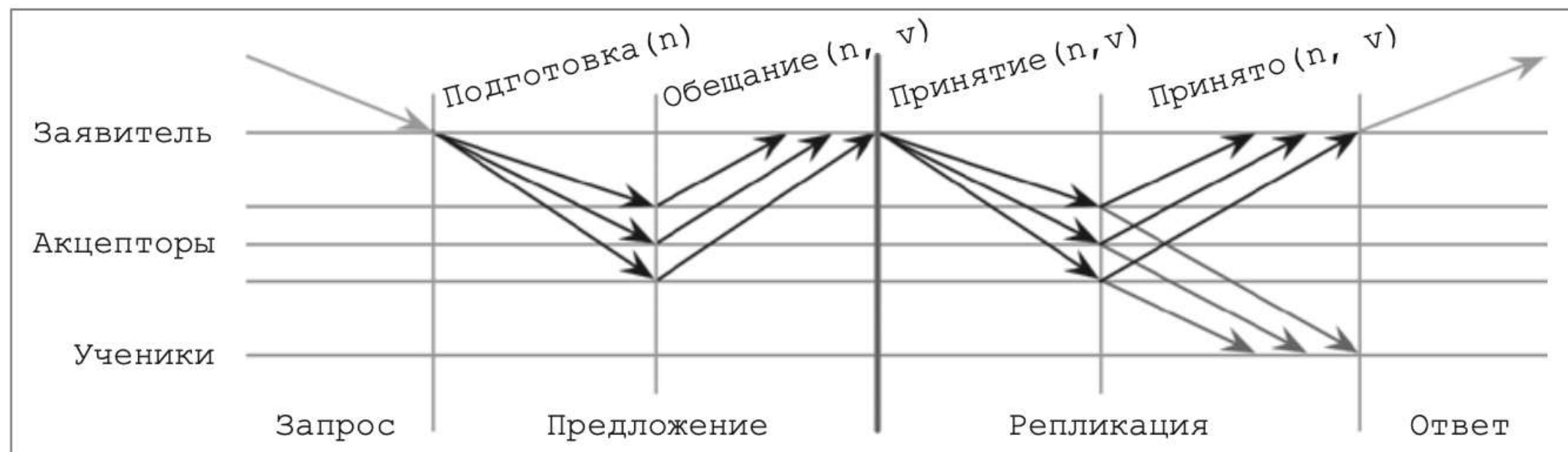
Разбор этапов алгоритма. Proposer stage 2

1. Proposer выбирает значение по предыдущей логике
2. И с выбранным значением отсылает сообщение Ассерпт!(n, v) большинству нод (или всем нодам)



Разбор этапов алгоритма. Acceptor stage 2

1. Акцептор получает значение в сообщении $\text{Accept!}(n, v)$
2. Если у Акцептора $n > n_{\text{promised}}$ – он принимает новое значение и отправляет $\text{Accepted}(n, v)$ обратно
3. Иначе – отправляет отвергает значение и отправляет обратно значение своего максимума

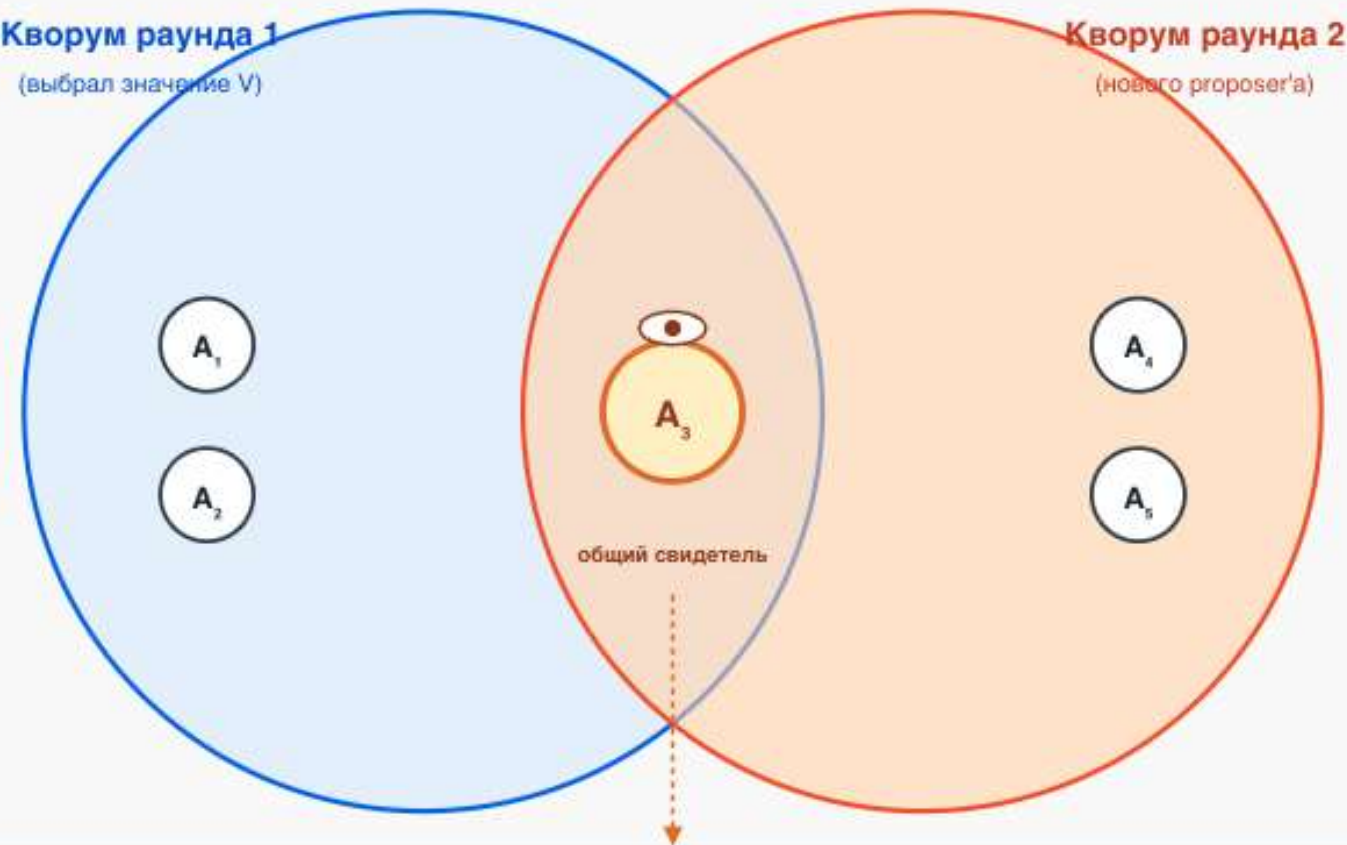


Кворумы в алгоритме Paxos

Сердце safety: пересечение большинств

Любые два кворума из $N=5$ имеют хотя бы один общий элемент

Кворум раунда 1
(выбрал значение V)



A_3 в Promise сообщит: «у меня уже принято V с номером n_a »

⇒ новый proposer ОБЯЗАН предложить V (правило $\max\text{-}n_a$)

⇒ Agreement сохранён ✓

Safety = пересечение большинств + правило $\max\text{-}n_a$

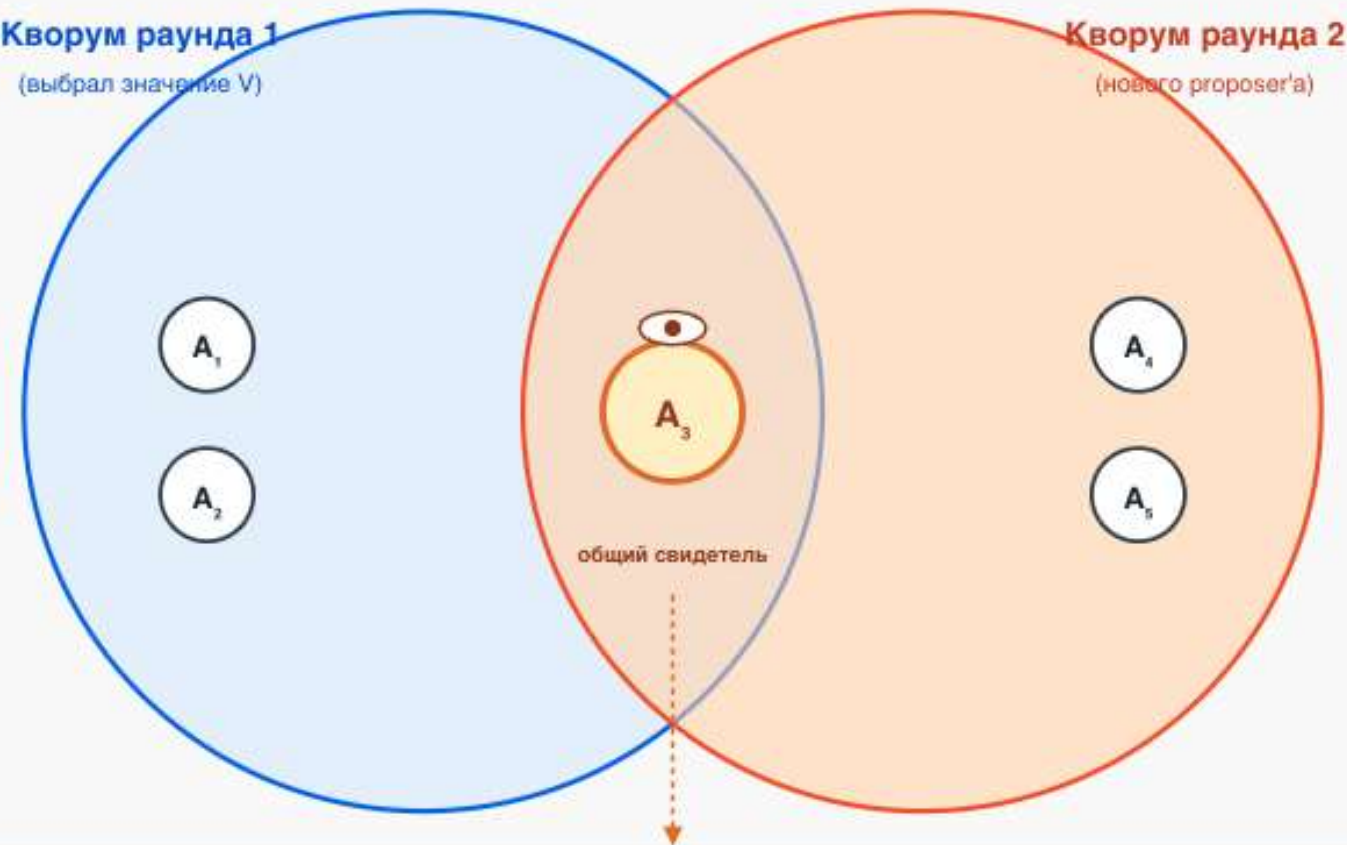
Кворумы в алгоритме Paxos

Сердце safety: пересечение большинства

Любые два кворума из $N=5$ имеют хотя бы один общий элемент

Кворум раунда 1

(выбрал значение V)



Кворум раунда 2

(нового proposer'a)

A_3 в Promise сообщит: «у меня уже принято V с номером n_a »

⇒ новый proposer ОБЯЗАН предложить V (правило $\text{max-}n_a$)

⇒ Agreement сохранён ✓

Safety = пересечение большинства + правило $\text{max-}n_a$